The Rust programming language

2023-08-29

Aleksandra

What is Rust?

- Compiled language
- Type safe
- Memory safe
  - No read/write out of bounds
  - No dangling references
- Manual memory management (*no garbage collection*)
- Prevents data races
- High performance

> ⓘ Note
>
> I think Rust is an interesting language to do a talk on because it's so
> different from most mainstream imperative/OO languages. The strict memory
> guarantees makes you approach problems in a different way in my experience.

> ⚠ Warning
>
> I am not an expert, I started learning this language for advent of code 2022,
> and I have only used it seriously for one research project at SINTEF in
> addition to playing around with the language for fun.

Syntax

```rust
#[derive(Clone, Debug)]
pub struct Graph {
    pub nodes: Vec<Node>,
    pub edges: Vec<Edge>,
}

impl Graph {
    pub fn is_preferred(&self, e1: &Edge, e2: &Edge) -> bool {
        if e1.cost < e2.cost {
            return true;
        }

        if e2.cost < e1.cost {
            return false;
        }

        self.edges.iter().position(|e| e == e1).expect("Edge 1 not in graph")
            < self.edges.iter().position(|e| e == e2).expect("Edge 2 not in graph")
    }

    pub fn adjacent_edges<'a>(&'a self, node: &'a Node) -> impl Iterator<Item = &'a
Edge> {
        self.edges.iter().filter(|&e| e.has_node(*node))
    }
}
```

Move by default

In Rust, assignments are move by default

- In C++ we have copy by default
- In C# we have reference by default
- In python we also have reference by default

```
1 let old_owner = String::new();   // ← old_owner owns created resource
2 let new_owner = old_owner;       // ← new_owner now owns this resource
3 println!("{}", &old_owner)       // ← ❗compile error! borrow after move
```

Compiling move.rs

```
error[E0382]: borrow of moved value: `old_owner`
 --> 01-move.rs:4:20
  |
2 |     let old_owner = String::from("Hello world");
  |         --------- move occurs because `old_owner` has type `String`, which does
not implement the `Copy` trait
3 |     let new_owner = old_owner;
  |                     --------- value moved here
4 |     println!("{}", old_owner);
  |                    ^^^^^^^^^ value borrowed here after move
```

Mutating variables

Variables in rust are immutable (const) by default

```
let str = String::new();
str.push_str("Hello world");
println!("{}", &str);
```

Compiling mutate.rs

```
error[E0596]: cannot borrow `str` as mutable, as it is not declared as mutable
 --> 02-mutate.rs:3:4
  |
3 |    str.push_str("Hello world");
  |    ^^^ cannot borrow as mutable
  |
help: consider changing this to be mutable
  |
2 |    let mut str = String::new();
  |        +++
```

Mutating variables

Variables in rust are immutable (const) by default

```
let mut str = String::new();
str.push_str("Hello world");
println!("{}", &str);  // Prints "Hello world"
```

To mutate them they need to be marked so using the mut keyword.

Borrowing

Taking a reference is called *borrowing*

You cannot mutate a borrowed variable

```rust
let mut a = String::from("Hello");
let b = &a;
a.push_str(" Optimisation!");
println!("{}", b); // Without this it compiles because it checks for future uses
```

Compiling mutate_borrow.rs

```
error[E0502]: cannot borrow `a` as mutable because it is also borrowed as immutable
 --> 03-mutate-borrow.rs:4:5
  |
3 |     let b = &a;
  |             -- immutable borrow occurs here
4 |     a.push_str(" Optimisation!");
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
5 |     println!("{}", b);
  |                    - immutable borrow later used here
```

Borrowing

Taking a reference is called *borrowing*

You cannot mutate a borrowed variable

```
let mut a = String::from("Hello");
let b = &mut a;
a.push_str(" Optimisation!");
println!("{}", b); // Without this it compiles because it checks for future uses
```

You also cannot borrow a variable as mutable more than once

```
error[E0499]: cannot borrow `a` as mutable more than once at a time
 --> 03-mutate-borrow.rs:4:5
  |
3 |     let b = &mut a;
  |             ------ first mutable borrow occurs here
4 |     a.push_str(" Optimisation!");
  |     ^ second mutable borrow occurs here
5 |     println!("{}", b);
  |                    - first borrow later used here
```

Structs

Structs are structured data containers as in e.g. C

```
struct ChatMessage {
    user: ChatUser,
    timestamp: DateTime<Utc>,
    message: String,
}

fn main() {
    let message = ChatMessage {
        user: ChatUser::new(),
        timestamp: Utc::now(),
        message: String::from("Hiya! o/"),
    }
}
```

Methods on structs

Although structs contain only data, one can declare methods for the structs, which as functions that has the struct as its context

```
impl ChatMessage {

    // Rust has no constructors, but it is common to name them new
    fn new(user: ChatUser, message: String) -> Self {
        Self {
            user: user,
            timestamp: Utc::now(),
            message: message
        }
    }

    // Reference self as mutable to change struct data
    fn correct_message(&mut self, new_message: String) {
        self.message = new_message
    }
}
```

Enums

Rust has *proper* enums, i.e. they are not just a set of numbers.

Rust enums can also wrap data.

```
enum Thing {
   City(String),
   Prime(u32),
   Colour(u8, u8, u8),
   Dog { name: String, species: DogSpecies }
}

enum DogSpecies {
   Dachs,
   GoldenRetriever,
   ...
}
```

The size of the enum object will then be the size of the option with the largest sized data.

Matching on enums

```
enum Thing {
   City(String),
   Prime(u32),
   // ...
}

// Matching with the match keyword
fn pretty_print_thing(thing: &Thing) {
    match thing {
        Thing::City(city_name) => println!("You are in {}", city_name),
        Thing::Prime(number) => println!("The prime number is {}", number),
        _ => println!("A thing!"),
    }
}

// Matching with if let
fn get_dog_species(thing: &Thing) -> Option<DogSpecies> {

   if let Thing::Dog { name: _, species} = thing {
       return Some(*species);
   }

   None   // No need for return statement on last line
}
```

Option<T> as "nullable"

Nullable in Rust is implemented as the `Option<T>` enum

```
enum Option<T> {
   Some(T),
   None
}
```

For instance a `Vec<T>` has a method `first()`

```
impl<T> Vec<T> {
   fn first(&self) -> Option<&T> {
       if self.is_empty() {
           return Option::None; // Can omit Option::
       }

       Option::Some(&self[0]) // Can omit Option::
   }
}
```

To use these values you need to *match* on them like other enums.

Option<T> as "nullable"

Nullable in Rust is implemented as the `Option<T>` enum

```
enum Option<T> {
   Some(T),
   None
}
```

There is also some helper syntax for transient calls using `?`

```
fn add_one_to_opt(value: Option<u8>) -> Option<u8> {
    Some(value? + 1)
}

fn main() {
   let none = add_one_to_opt(None);    // none = None
   let some = add_one_to_opt(Some(8)); // some = Some(9)
}
```

Exceptions with Result<T,E>

Exceptions in Rust are implemented as the `Result<T,E>` enum

```
enum Result<T,E> {
   Ok(T),
   Err(E)
}
```

There is no throwing and propagation up the call stack, the errors must always be handled or passed.

In addition to matching them as enums they also have helpers to unwrap the result

```
let value = Ok(42);
value.unwrap();                 // panics if value is Err
value.unwrap_or(12);            // returns 12 is value is Err
value.expect("Expect no errors"); // Unwrap with custom error message
```

There is also some helper syntax for transient calls using `?`

```
fn main() -> Result<(), Box<dyn Error>> {
    let file = File::open("file")?;

    Ok(())
}
```

Traits

Traits are the closet we come to interfaces in Rust, but they are more akin to concepts in C++

```
trait FormatMessage {
    fn to_message(&self) -> String;
}
```

These can then be implemented for your struct

```
impl FormatMessage for ChatMessage {
    fn to_message(&self) -> String {
        format!(
            "{}  @{} | {}",
            self.timestamp.time(),
            self.user.username,
            self.message
        )
    }
}
```

Traits

Traits are the closet we come to interfaces in Rust, but they are more akin to concepts in C++

```
trait FormatMessage {
    fn to_message(&self) -> String;
}
```

You can also extend any type with this trait, including base types

```
impl FormatMessage for &str {
    fn to_message(&self) -> String {
        String::from(*self)
    }
}

impl FormatMessage for u32 {
    fn to_message(&self) -> String {
        self.to_string()
    }
}

impl FormatMessage for String {
    fn to_message(&self) -> String {
        self.clone()
    }
}
```

Traits

Traits are the closet we come to interfaces in Rust, but they are more akin to concepts in C++

```
trait FormatMessage {
    fn to_message(&self) -> String;
}
```

None of these have to be defined at the same place in the code. So you can

- Implement traits for your structs elsewhere
- Extend external structs with new traits

In that sense they are similar to extension methods in C#

Templates

Templates in rust must be fully specified with respect to type traits

```
fn add_message_to_queue<M>(queue: &mut Vec<String>, message: &M)
where
    M: FormatMessage,
{
    queue.push(message.to_message())
}
```

This function can be called with any type `M` that implements the `FormatMessage` trait

Traits can also be composed

```
fn some_complex_function<M>(message: &M)
where
   M: FormatMessage + IntoIterator + PartialOrd
{
    // ...
}
```

Meaning that you do not need to define new traits that is simply a composition of others.

Lifetime annotations

Sometimes when passing references to functions we need to tell the compiler the lifetimes of the references passed if it can't be inferred (*it often can*)

```
fn say_hello<'lifetime1, 'lifetime2>(
    user1: &'lifetime1 ChatUser,
    user2: &'lifetime2 ChatUser,
) -> ChatMessage<'lifetime1> {
    ChatMessage::new(user1, format!("Hello {}! :3", user2.username))
}
```

If it's runtime dependent then we have to assume that both borrows has the longest lifetime of the two

```
fn say_hello_to_shortest_user<'lifetime>(
    user1: &'lifetime ChatUser,
    user2: &'lifetime ChatUser,
) -> ChatMessage<'lifetime> {

    if user1.username < user2.username {
        return ChatMessage::new(user1, format!("Hello {}! :3", user2.username))
    }

    ChatMessage::new(user2, format!("Hello {}! <3", user1.username))
}
```

Lambdas

Lambdas are similat to other languages, but remember that they also need ownership and reference specifications

```
// A lambda that borrows a str value
let lambda = |x: &str| -> usize { x.len() };

// A lambda that consumes its argument
let lambda = |username: String| -> ChatMessage { ChatMessage::new(username, "") };
```

Closures

Closures work as expected, to make sure the ownership rules are preserved, function objects implement one of the following traits

- `FnOnce`: Functions that can only be called once because it consumes its arguments.
- `FnMut`: Functions that mutate its captured values.
- `Fn`: Functions that neither move nor mutate its captured values.

 FnOnce

```
let mut items = vec![];
let text = String::from("foo");

let closure = || {
    items.push(text);
};

closure();
```

Closures

Closures work as expected, to make sure the ownership rules are preserved, function objects implement one of the following traits

- `FnOnce`: Functions that can only be called once because.
- `FnMut`: Functions that mutate its captured values.
- `Fn`: Functions that neither move nor mutate its captured values.

 FnMut

```
let mut items = vec![];

let closure = |text: &str| {
    items.push(String::from(text));
};

closure("foo");
```

Closures

Closures work as expected, to make sure the ownership rules are preserved, function objects implement one of the following traits

- `FnOnce`: Functions that can only be called once because.
- `FnMut`: Functions that mutate its captured values.
- `Fn`: Functions that neither move nor mutate its captured values.

 Fn

```
let text = String::from("foo");

let closure = || {
    format!("Hello {}", text)
};

closure();
```

Iterators

Iterators in rust are *lazy*, meaning that they have no effect until called.

```
let items = vec!["Hello", "world", "foo", "bar", "baz"];
let value = items
    .iter()
    .filter(|&s| s.len() == 3)
    .map(|&s| s.to_uppercase())
    .reduce(|acc, s| format!("{} - {}", acc, s));
```

They allow the type of chained transformations common in functional programming / fluent syntax / ...

Cargo

Cargo is Rust's build system and package manager.

Most Rust projects user Cargo to handle all these tasks, such as

- Building the project
- Downloading dependencies
- Running tests
- Defining build options and targets
- Composing larger projects

Project settings are defined in `Cargo.toml` at the root of your project

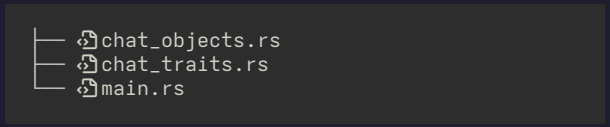
```
cargo new [project name]       # Create a new rust project
cargo add [dependency name]    # Add new external dependency
cargo build                    # Build the project
cargo run                      # Run the project executable
cargo test                     # Run all tests in the project
```

Structuring Rust projects

Some terminology

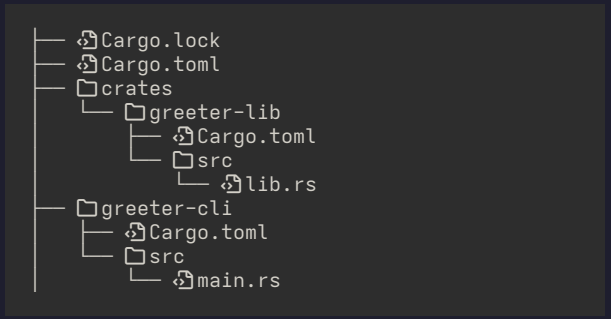
- A *module* is a collection of Rust items like functions, structs, traits, ...
- A *crate* is a compilation unit that results in either a library or an executable
- A *project* is a collection of crates that can have multiple executables, but at most one library
- A *workspace* is a collection of projects that usually depend on each other

Structuring Rust projects (modules)




```rust
// In src/main.rs
use crate::chat_traits::ChatMessage;

// Necessary to include the file as a port of the compilation unit
mod chat_traits;
mod chat_objects;


// In src/chat_objects.rs
use crate::chat_traits::FormatMessage;

pub struct ChatMessage { ... }

impl FormatMessage for ChatMessage { ... }


// In src/chat_traits.rs
pub trait FormatMessage { ... }
```

Structuring Rust projects (workspaces)




```toml
# Cargo.toml
# ---
[workspace]
members = [
  "crates/*",
  "greeter-cli"
]

# greeter-cli/Cargo.toml
# ---
[dependencies]
greeter_lib = { path = "../crates/greeter-lib" }
```

Questions

Talk is available at **github.com/irubataru/talks** (*https://github.com/irubataru/talks*)

Additional resources

- **The Rust book** (*https://doc.rust-lang.org/book/*)
  - Basic language introduction and syntax
- **The Rust reference** (*https://doc.rust-lang.org/reference*)
  - Descriptions of the language constructs
- **The Rustonomicon** (*https://doc.rust-lang.org/nomicon*)
  - The nitty gritty details
- Look at popular open soruce Rust projects for inspiration!

Lambdas & type inference

There is also type inference so you don't need to specify the type if it can be inferred. This is very common in iterator predicates as we will see.

Beware that only one type can be inferred for a single lambda

```
let passthrough = |x| x;
let text = passthrough(String::from("Hello world"));
let number = passthrough(42); // ❗Compile error, only one type can be inferred
```

Compiling lambda.rs

```
error[E0308]: mismatched types
 --> 10-lambda.rs:4:30
  |
4 |     let number = passthrough(42); // Compile error, only one type can be
inferred
  |                  ----------- ^^- help: try using a conversion method:
`.to_string()`
  |                  |           |
  |                  |           expected `String`, found integer
  |                  arguments to this function are incorrect
```

More complicated tempate example

```rust
pub fn smallest<'a, I, T>(mut iter: I) -> Option<&'a T>
where
    I: Iterator<Item = &'a T>,
    T: PartialOrd
{
    let mut result: Option<&'a T> = None;

    while let Some(item) = iter.next() {
        result = match result {
            Some(value) => {
                if item < value {
                    Some(item)
                } else {
                    Some(value)
                }
            },
            None => Some(item),
        };
    }

    result
}
```

`PartialOrd` means that the items can be compared with `<` and other ordering operators

Extending Iterator

Testing

Heap objects

Runtime memory management